

Pairs and Stripes

Mahmoud Parsian

Ph.D. in Computer Science

Table of Contents

1. The Problem
2. Two Ways to Emit the Same Answer
3. Example 1: Word Co-occurrence
4. Final Co-occurrence Matrix & Tradeoffs
5. Example 2: Products Bought Together
6. Recap
7. References

1. The Problem

- You have a huge collection of documents.
- For every pair of things that appear together, you want a count: how often did they co-occur?
- The result is an $N \times N$ co-occurrence matrix — built with MapReduce, never all in memory at once.
- Shows up everywhere: word co-occurrence, "customers who bought X also bought Y", "users who follow A also follow B".

2. Two Ways to Emit the Same Answer

- **Pairs** — emit one `(word1, word2) -> 1` record per co-occurrence. Let the reducer sum them.
- **Stripes** — emit one `word -> {other_word: count, ...}` record per word. Let the reducer merge the maps.

Same final answer. Very different mapper output size, shuffle traffic, and reducer memory.

3. Example 1: Word Co-occurrence

Tiny 3-line corpus. Neighbors = every other word on the same line.

```
Line 1: cat sat mat  
Line 2: cat sat rug  
Line 3: sat mat rug
```

We'll solve this same corpus twice: once with Pairs, once with Stripes.

Pairs — Mapper

For each line, emit every ordered word pair with count 1:

```
map(key, line):  
    for w1 in line:  
        for w2 in line:  
            if w1 != w2:  
                emit((w1, w2), 1)
```

A MapReduce mapper always receives `(key, value)` input — here `key` is just the line number (or byte offset) Hadoop/Spark hands you for free, and `value` is the line's text. This algorithm never uses `key`; only `line` matters.

Pairs — Mapper (Continued)

Line 1 emits:

```
(cat,sat) (cat,mat) (sat,cat) (sat,mat) (mat,cat) (mat,sat)
```

Pairs — All Mapper Output

```
Line 1: (cat,sat)=1 (cat,mat)=1 (sat,cat)=1 (sat,mat)=1 (mat,cat)=1 (mat,sat)=1  
Line 2: (cat,sat)=1 (cat,rug)=1 (sat,cat)=1 (sat,rug)=1 (rug,cat)=1 (rug,sat)=1  
Line 3: (sat,mat)=1 (sat,rug)=1 (mat,sat)=1 (mat,rug)=1 (rug,sat)=1 (rug,mat)=1
```

18 key-value records cross the shuffle — one per pair, per line.

Every single record is listed above, nothing skipped.

Pairs — Reducer

Shuffle groups all 18 records by key into 12 distinct keys. Reducer sums the 1's for each key — `cat` 's and `sat` 's rows first:

```
reduce((cat,sat), [1,1]) -> 2
reduce((cat,mat), [1])   -> 1
reduce((cat,rug), [1])   -> 1
reduce((sat,cat), [1,1]) -> 2
reduce((sat,mat), [1,1]) -> 2
reduce((sat,rug), [1,1]) -> 2
```

Pairs — Reducer (Continued)

mat 's and rug 's rows — all 12 keys now fully reduced:

```
reduce( (mat,cat), [1] )    -> 1
reduce( (mat,sat), [1,1] ) -> 2
reduce( (mat,rug), [1] )    -> 1
reduce( (rug,cat), [1] )    -> 1
reduce( (rug,sat), [1,1] ) -> 2
reduce( (rug,mat), [1] )    -> 1
```

Every reducer output is listed above — 12 keys, nothing skipped.

Stripes — Mapper

Same corpus, but each word emits **one** record per line: itself, mapped to a small count-map of its neighbors on that line.

```
map(key, line):  
    for w1 in line:  
        stripe = {}  
        for w2 in line:  
            if w1 != w2:  
                stripe[w2] += 1  
        emit(w1, stripe)
```

Same as before: `key` (the line number) is ignored — only `line` drives the logic.

Stripes — All Mapper Output

```
Line 1: cat->{sat:1,mat:1}  sat->{cat:1,mat:1}  mat->{cat:1,sat:1}
Line 2: cat->{sat:1,rug:1}  sat->{cat:1,rug:1}  rug->{cat:1,sat:1}
Line 3: sat->{mat:1,rug:1}  mat->{sat:1,rug:1}  rug->{sat:1,mat:1}
```

9 records cross the shuffle, not 18 — each one just carries more data. Every single record is listed above, nothing skipped.

Stripes — Reducer

The reducer merges every stripe it receives for a key, adding matching entries together — `cat` 's and `sat` 's rows first:

```
reduce(cat, [{sat:1,mat:1}, {sat:1,rug:1}])  
  -> {sat:2, mat:1, rug:1}
```

```
reduce(sat, [{cat:1,mat:1}, {cat:1,rug:1}, {mat:1,rug:1}])  
  -> {cat:2, mat:2, rug:2}
```

Stripes — Reducer (Continued)

mat 's and rug 's rows — all 4 keys now fully reduced:

```
reduce(mat, [{cat:1,sat:1}, {sat:1,rug:1}])  
  -> {cat:1, sat:2, rug:1}  
  
reduce(rug, [{cat:1,sat:1}, {sat:1,mat:1}])  
  -> {cat:1, sat:2, mat:1}
```

Every reducer output is listed above — 4 keys, nothing skipped.

4. Final Co-occurrence Matrix & Tradeoffs

Pairs (12 keys) and Stripes (4 keys) agree exactly — same matrix, built two different ways:

Word	Co-occurs with (count)
cat	sat:2, mat:1, rug:1
sat	cat:2, mat:2, rug:2
mat	cat:1, sat:2, rug:1
rug	cat:1, sat:2, mat:1

The Real Tradeoff

	Pairs	Stripes
Mapper output	many, tiny records	few, larger records
Shuffle traffic	high — w1 repeated for every neighbor	low — w1 written once per line
Combiner benefit	small, per-pair	large, whole maps merge
Reducer memory	trivial (one counter)	can be large (whole row)

A word with 1,000 distinct neighbors: Pairs emits 1,000 tiny records; Stripes emits 1 record holding a 1,000-entry map.

Why Stripes Usually Wins

- Combiners work far better on Stripes: merging two maps for the same word, before shuffle, removes almost all duplicate keys.
- Fewer, larger records shuffle faster than many tiny ones — per-record network overhead adds up.
- Downside: one very frequent word (e.g. "the") builds one huge stripe and can blow reducer memory. Real systems cap stripe size, or fall back to Pairs for the most frequent words.

5. Example 2: Products Bought Together

Same question, different domain: for every pair of products, how often were they bought in the same order?

```
Order 1: bread milk eggs  
Order 2: bread eggs  
Order 3: milk eggs cheese
```

Example 2 — Mapper, Worked

```
map(01): bread->{milk:1,eggs:1}  milk->{bread:1,eggs:1}  eggs->{bread:1,milk:1}  
map(02): bread->{eggs:1}  eggs->{bread:1}  
map(03): milk->{eggs:1,cheese:1}  eggs->{milk:1,cheese:1}  cheese->{milk:1,eggs:1}
```

7 records total — one per item per order. Every record is listed above, nothing skipped.

Example 2 — Reducer, Worked

```
reduce(bread, [{milk:1,eggs:1}, {eggs:1}])  
  -> {milk:1, eggs:2}  
reduce(milk, [{bread:1,eggs:1}, {eggs:1,cheese:1}])  
  -> {bread:1, eggs:2, cheese:1}  
reduce(eggs, [{bread:1,milk:1}, {bread:1}, {milk:1,cheese:1}])  
  -> {bread:2, milk:2, cheese:1}  
reduce(cheese, [{milk:1,eggs:1}])  
  -> {milk:1, eggs:1}
```

All 4 products reduced — **eggs** was bought with bread twice, milk twice, cheese once.

6. Recap

- Both techniques answer the same question: for every pair of things that co-occur, what's the count?
- **Pairs**: simple, one tiny record per pair — the shuffle and combiner do all the work.
- **Stripes**: one record per item per document, carrying a small map — fewer, richer records, bigger combiner win, bigger reducer memory risk.
- Works for word co-occurrence, co-purchased products, mutual followers, co-watched movies — anywhere you need an $N \times N$ "who appears with whom" matrix.

7. References

1. Jimmy Lin & Chris Dyer — "Basic MapReduce Algorithm Design" (Pairs and Stripes), from *Data-Intensive Text Processing with MapReduce* — [PDF](#)
2. *Data Algorithms* — Mahmoud Parsian
3. *Data Algorithms with Spark* — Mahmoud Parsian